30 SEP 2024 • 4 MIN READ • CONTAINERS

How to Use Tailscale Serve with Docker Compose for Secure, Private Self-Hosting

Learn how to securely self-host services using Tailscale Serve without exposing them to the public internet. This guide covers setups for Uptime Kuma and Windmill, with step-by-step examples and tips for Docker users to simplify infrastructure and improve privacy.

I've been moving a lot of my self hosting infra over to tailscale serve recently. It's an absolute dream... once you get it working. This sort of use case seems to be lacking in any real coherent articles or documentation, so I thought I'd put together a few examples and notes for anyone who's looking to do this themselves.

Why?

Before jumping into the examples, I think it's worth briefly explaining why I like this setup. If this isn't something you need, feel free to skip down to the examples.

I'm a big fan of keeping things simple. I host a number of services for myself and my family, many of which use languages or platforms I'm not very familiar with (PHP, for example). Docker makes this easy and is something I know well, and docker compose is perfect for this sort setup. I don't need five nines of uptime, blue-green deploys, or horizontal scaling. Most of my services live on a dedicated hetzner server with plenty of resources, and the rest sit happily on a TinyMiniMicro PC.

Most of these services just don't need to accept external traffic, so there's really no need to expose them to the public internet. I'm already using tailscale for ssh access and networking

between these devices, so using it for making each service available privately over HTTP just made sense for me.

I still have public services (like this blog), those don't use tailscale at all and are exposed the old fashioned way.

Uptime Kuma - A simple example

This example uses tailscale serve to make uptime kuma accessible on your tailnet. It's dead simple, just two containers!

```
services:
  web:
    image: louislam/uptime-kuma:1.23.11
    depends_on:
      - tailscale
    restart: unless-stopped
    network_mode: service:tailscale
    environment:
      - UPTIME_KUMA_PORT=80
    volumes:
      - data:/app/data

  tailscale:
    image: ghcr.io/tailscale/tailscale:v1.74.1
    restart: unless-stopped
    environment:
      - TS_AUTHKEY={{ secrets.tailscale_authkey }}?ephemeral=true
      - TS_EXTRA_ARGS=--advertise-tags=tag:container,tag:internal-app
      - TS_SERVE_CONFIG=/config/ts-serve.json
      - TS_HOSTNAME=uptime
    volumes:
      - "./tailscale_var_lib:/var/lib"
      - "./config:/config"
      - "/dev/net/tun:/dev/net/tun"
    cap_add:
      - net_admin
      - sys_module
```

```
  volumes:
    data:
```

This is a pretty small example, but none the less I'll explain the fundamentals.

First we have our uptime kuma container, this isn't configured in any particular way really. The app is running on port 80, but this port isn't exposed outside the container as tailscale serve will proxy traffic on.

The more important bit is the network setting, we have uptime kuma sharing the network with the tailscale container. I'm not completely sure *why* we need to do this, but it does seem to be required - otherwise the forwarding (even when changing hostnames around) doesn't seem to work.

Then we have our tailscale container, this is mostly ripped from the example on the tailscale site. You can see I pass in an auth key, make the container ephemeral, and some tags to apply to the container. You don't really need the `./tailscale_var_lib:/var/lib` mount, but I've found it helps speed up restarts. You will need `./config:/config` as that config directory will contain your `ts-serve.json` file which configures the tailscale serve.

That `ts-serve.json` file contains the following. You should be able to use this for basically any service which is just a standard HTTP service listening on port 80. The `${TS_CERT_DOMAIN}` is something tailscale just handles and subsitutes for the tailscale fully qualified domain of the tailscale node.

```
{
    "TCP": {
        "443": {
            "HTTPS": true
        }
    },
    "Web": {
        "${TS_CERT_DOMAIN}:443": {
            "Handlers": {
                "/": {
                    "Proxy": "http://127.0.0.1:80"
                }
            }
```

```
        }
    },
    "AllowFunnel": {
        "${TS_CERT_DOMAIN}:443": false
    }
}
```

You may also notice that in my docker-compose.yml file I've got a few things like `{{ secrets.tailscale_authkey }}`. This is because I template my compose files using ansible and inject those secrets in when coping over the file. You can just replace in your relevant values here. Maybe at some point I'll write up how I use ansible to manage all these compose files.

Windmill - A more complex example

The principals are all the same with this, just with more containers, and more ports we need to handle. In this case there's port 80/443 for the main HTTP service, and then port 3001 for websockets, and 3002 for multiplayer websockets.

Aside from that, everything is pretty much the same - but here's the full example so you can see it.

```
services:
  db:
    image: postgres:16
    restart: unless-stopped
    volumes:
      - db_data:/var/lib/postgresql/data
    environment:
      POSTGRES_PASSWORD: {{ secrets.postgres_password }}
      POSTGRES_DB: {{ secrets.postgres_database }}
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
    network_mode: service:tailscale
```

```
    windmill_server:
      image: ghcr.io/windmill-labs/windmill:1.402.2
      pull_policy: always
      restart: unless-stopped
      environment:
        - DATABASE_URL=postgres://postgres:{{ secrets.postgres_password }}@127.
        - MODE=server
      depends_on:
        db:
          condition: service_healthy
      network_mode: service:tailscale

    windmill_worker:
      image: ghcr.io/windmill-labs/windmill:1.402.2
      pull_policy: always
      restart: unless-stopped
      environment:
        - DATABASE_URL=postgres://postgres:{{ secrets.postgres_password }}@127.
        - MODE=worker
        - WORKER_GROUP=default
      depends_on:
        db:
          condition: service_healthy
      # to mount the worker folder to debug, KEEP_JOB_DIR=true and mount /tmp/w
      volumes:
        # mount the docker socket to allow to run docker containers from within
        - /var/run/docker.sock:/var/run/docker.sock
        - worker_dependency_cache:/tmp/windmill/cache
      network_mode: service:tailscale

    ## This worker is specialized for "native" jobs. Native jobs run in-process
    windmill_worker_native:
      image: ghcr.io/windmill-labs/windmill:1.402.2
      pull_policy: always
      restart: unless-stopped
      environment:
        - DATABASE_URL=postgres://postgres:{{ secrets.postgres_password }}@127.
        - MODE=worker
        - WORKER_GROUP=native
      depends_on:
        db:
```

```
        condition: service_healthy
    network_mode: service:tailscale

  lsp:
    image: ghcr.io/windmill-labs/windmill-lsp:1.402.2
    pull_policy: always
    restart: unless-stopped
    volumes:
      - lsp_cache:/root/.cache
    network_mode: service:tailscale

  tailscale:
    image: ghcr.io/tailscale/tailscale:v1.74.1
    restart: unless-stopped
    environment:
      - TS_AUTHKEY={{ secrets.tailscale_authkey }}?ephemeral=true
      - TS_EXTRA_ARGS=--advertise-tags=tag:container,tag:internal-app
      - TS_SERVE_CONFIG=/config/ts-serve.json
      - TS_HOSTNAME=windmill
    volumes:
      - "./tailscale_var_lib:/var/lib"
      - "./config:/config"
      - "/dev/net/tun:/dev/net/tun"
    cap_add:
      - net_admin
      - sys_module

volumes:
  db_data:
    driver: local
  worker_dependency_cache:
    driver: local
  lsp_cache:
    driver: local
```

And the `ts-serve.json`:

```
{
    "TCP": {
        "443": {
            "HTTPS": true
        },
        "3001": {
            "HTTPS": true
        },
        "3002": {
            "HTTPS": true
        }
    },
    "Web": {
        "${TS_CERT_DOMAIN}:443": {
            "Handlers": {
                "/": {
                    "Proxy": "http://127.0.0.1:8000"
                }
            }
        },
        "${TS_CERT_DOMAIN}:3001": {
            "Handlers": {
                "/ws/*": {
                    "Proxy": "http://127.0.0.1:3001"
                }
            }
        },
        "${TS_CERT_DOMAIN}:3002": {
            "Handlers": {
                "/ws_mp/*": {
                    "Proxy": "http://127.0.0.1:3002"
                }
            }
        }
    },
    "AllowFunnel": {
        "${TS_CERT_DOMAIN}:443": false,
        "${TS_CERT_DOMAIN}:3001": false,
        "${TS_CERT_DOMAIN}:3002": false
    }
```

Captured by FireShot Pro: 03 March 2026, 04:17:53 PM
https://getfireshot.com

```
  }
```

Once again, the main things here are the fact we're not exposing any ports, all services are sharing the tailscale containers network, and the `ts-serve.json` is mounted for tailscale serve to use.

I hope this helps you get up and running quicker than I did. If you figure out any improvements, get in touch and let me know!

Published by:

You might also like...

JUL 04	Tailscale for lazy application authentication	6 min read
MAY 08	"certificate not yet valid" when installing tailscale via cloud-init on Raspberry ...	2 min read
AUG 31	DKIM, SPF, MX, and other email DNS records	7 min read
NOV 16	Just Eat's engineering rituals	5 min read
MAY 08	How I use git	5 min read

Member discussion

2 comments

Join the discussion

Become a member of **Elliot Blackburn** to start commenting.

Sign up now

Already a member? Sign in

Sort by: Best

A **A4alii** · 9 Nov 2024

It would be more interesting if you make a tutorial about

1. Ente photos with tailscale

2. Snikket or prosody with tailscale

I tried both but failed, due to lack of experience and knowledge

0 Reply

A **A4alii** · 9 Nov 2024

Amazing. I'm recently running nextcloud-aio using caddy as sidecar and tailscale.

https://github.com/nextcloud/all-in-one/discussions/5439

May be if you lookup, BTW my apache container is unhealthy with docker rootless

0 Reply

Elliot Blackburn © 2026

Powered by Ghost